

DraftProAnalytics Git and VPS Deployment Procedure

1. General rules

1. All new development must occur on a feature branch.
 2. Do not develop directly on `main`.
 3. Do not copy local `.env` files into GitHub.
 4. Do not overwrite the VPS `.env` files during deployment.
 5. The VPS must deploy only committed code from GitHub `main`.
 6. Merge into `main` only after the feature branch has been tested successfully.
 7. Before deploying, verify that `main` has been pushed to GitHub.
-

Part I — Local development procedure

2. Check out the latest GitHub `main` branch

Perform these steps separately in the client and server repositories.

Client

```
cd ~/aiAssistWS/draftProAnalytics-client

git status
git fetch origin
git switch main
git pull --ff-only origin main
```

Server

```
cd ~/aiAssistWS/draftProAnalytics-server

git status
git fetch origin
git switch main
git pull --ff-only origin main
```

The `--ff-only` option prevents Git from creating an unexpected merge commit during the pull.

Verify the current branch:

```
git branch --show-current
```

Expected result:

```
main
```

Verify that the working directory is clean:

```
git status
```

Expected result:

```
nothing to commit, working tree clean
```

3. Create a feature branch

Use a descriptive branch name.

Recommended format:

```
feature/<feature-name>
```

Examples:

```
feature/draft-pattern-analysis  
feature/import-weekly-scores  
feature/admin-user-management
```

Create the branch from the latest `main`:

```
git switch -c feature/<feature-name>
```

Example:

```
git switch -c feature/import-weekly-scores
```

Verify the branch:

```
git branch --show-current
```

4. Develop and commit the feature

Review changes:

```
git status  
git diff
```

Stage the changes:

```
git add .
```

Review what will be committed:

```
git diff --cached
```

Commit the changes:

```
git commit -m "Add weekly score import feature"
```

Push the feature branch to GitHub:

```
git push -u origin feature/<feature-name>
```

After the first push, later updates can use:

```
git push
```

5. Keep the feature branch current with `main`

Before final testing and merging, update local `main`:

```
git switch main
git fetch origin
git pull --ff-only origin main
```

Switch back to the feature branch:

```
git switch feature/<feature-name>
```

Merge current `main` into the feature branch:

```
git merge main
```

If Git reports conflicts:

```
git status
```

Resolve each conflicted file, then run:

```
git add <resolved-file>
git commit
```

Push the updated feature branch:

```
git push
```

Do not merge the feature branch into `main` until all conflicts are resolved and testing succeeds.

6. Test the feature branch

Client testing

```
cd ~/aiAssistWS/draftProAnalytics-client
```

```
npm install
npm run type-check
npm run build
```

If the project does not have a `type-check` script, use:

```
npx vue-tsc --noEmit
```

Run the development client when needed:

```
npm run dev
```

Server testing

```
cd ~/aiAssistWS/draftProAnalytics-server

npm install
npm run build
```

Run tests if configured:

```
npm test
```

Run the development server when needed:

```
npm run dev
```

Before merging, verify both repositories:

```
git status
```

The feature branch should have no uncommitted changes.

Part II — Merge the feature branch into `main`

7. Update local `main`

```
git switch main
git fetch origin
git pull --ff-only origin main
```

8. Check whether the merge is likely to conflict

From `main`, run:

```
git merge-tree "$(git merge-base main feature/<feature-name>)" main feature/
<feature-name>
```

You may also perform a test merge without creating a commit:

```
git merge --no-commit --no-ff feature/<feature-name>
```

If Git reports no conflicts, cancel the test merge before performing the formal merge:

```
git merge --abort
```

If `git merge --abort` says there is no merge to abort, no temporary merge state exists.

9. Merge the feature branch into `main`

```
git switch main
git merge --no-ff feature/<feature-name>
```

Git may open the Nano editor for the merge commit message.

In Nano:

```
Ctrl+O    Save
Enter     Confirm filename
Ctrl+X    Exit
```

Verify the result:

```
git status
git log --oneline --graph --decorate -10
```

Push `main` to GitHub:

```
git push origin main
```

10. Optionally delete the completed feature branch

Delete the local branch:

```
git branch -d feature/<feature-name>
```

Delete the GitHub branch:

```
git push origin --delete feature/<feature-name>
```

Only delete the branch after confirming that `main` contains the completed feature.

Part III — Connect to the VPS

11. SSH into DomaniRacer.com VPS

```
ssh <vps-user>@104.225.219.35
```

Example:

```
ssh dthompson@104.225.219.35
```

Use the actual VPS username configured on the server.

After login, verify the host and current directory:

```
hostname  
pwd
```

Part IV — Server deployment

12. Navigate to the server application

```
cd /var/www/draftproanalytics-server
```

Verify the repository:

```
git status  
git remote -v  
git branch --show-current
```

13. Protect the VPS environment file

Confirm that `.env` exists:

```
ls -la .env
```

Create a backup before deployment:

```
cp .env "$HOME/draftproanalytics-server.env.backup"
```

Confirm that `.env` is ignored by Git:

```
git check-ignore .env
```

Expected output:


```
.env
```

The repository should contain an example file such as:

```
.env.example
```

The production `.env` file must not be tracked by Git.

Check whether Git tracks `.env`:

```
git ls-files .env
```

Expected output:

There should be no output.

If `.env` appears in the output, stop the deployment. The file is incorrectly tracked by Git.

14. Check for uncommitted VPS changes

```
git status
```

The VPS working tree should normally be clean.

Do not use this command when important uncommitted VPS changes exist:

```
git reset --hard
```

First inspect the differences:

```
git diff
git status --short
```

Production source-code changes should not be made directly on the VPS.

15. Check out the GitHub `main` branch

```
git fetch origin
git switch main
git pull --ff-only origin main
```

For a stricter deployment that makes the VPS code exactly match GitHub `main`:

```
git fetch origin
git switch main
git reset --hard origin/main
```

This does not delete an untracked `.env` file.

However, never run:

```
git clean -fd
```

without reviewing the untracked files first, because it can delete production files that are not committed.

To preview what Git would delete:

```
git clean -nd
```

16. Install server dependencies

For a reproducible install:

```
npm ci
```

If the server build requires development dependencies, do not use `--omit=dev` before building.

17. Apply database changes when required

Use the approved MySQL-first deployment procedure for schema changes.

Do not automatically run Prisma migrations unless the release specifically includes and approves them.

If Prisma Client changed, regenerate it:

```
npx prisma generate
```

18. Build the server

```
npm run build
```

Confirm that the build completed successfully before restarting PM2.

Typical build output directory:

```
ls -la dist
```

19. Restart the PM2 server process

First inspect the PM2 process list:

```
pm2 list
```

Restart using the configured process name:

```
pm2 restart draftproanalytics-server --update-env
```

If your PM2 process has a different name, use the name shown by:

```
pm2 list
```

If the project uses an ecosystem file:

```
pm2 restart ecosystem.config.js --only draftproanalytics-server --update-env
```

Save the PM2 process configuration:

```
pm2 save
```

20. Verify the server deployment

Inspect status:

```
pm2 status
```

Inspect recent logs:

```
pm2 logs draftproanalytics-server --lines 100
```

Exit the live log display with:

```
Ctrl+C
```

Test the local server from the VPS:

```
curl -i http://127.0.0.1:5000/api/health
```

Use an actual known endpoint if `/api/health` is not implemented:

```
curl -i http://127.0.0.1:5000/api/access/me
```

A protected endpoint may return `401 Unauthorized`; that still confirms that nginx or Express is responding.

Check the public API through nginx:

```
curl -i https://draftproanalytics.com/api/health
```

Part V — Client deployment

21. Navigate to the client application

```
cd /var/www/draftproanalytics-client
```

Verify the repository:

```
git status
git remote -v
git branch --show-current
```

22. Protect the client production environment file

Confirm that the client environment file exists:

```
ls -la .env*
```

Common production files include:

```
.env
.env.production
```

Create backups:

```
test -f .env && cp .env "$HOME/draftproanalytics-client.env.backup"
test -f .env.production && cp .env.production "$HOME/draftproanalytics-
client.env.production.backup"
```

Check whether Git tracks them:

```
git ls-files .env .env.production
```

Expected output:

There should be no output.

Example files may remain tracked:

```
.env.example
.env.production.example
```

23. Check out GitHub `main`

```
git fetch origin
git switch main
git pull --ff-only origin main
```

For a strict deployment:

```
git fetch origin
git switch main
git reset --hard origin/main
```

Again, do not run `git clean -fd` without reviewing:

```
git clean -nd
```

24. Install client dependencies

```
npm ci
```

25. Build the Vue client

```
npm run build
```

For a Vite application, the production files normally appear under:

```
ls -la dist
```

The Vue client itself normally does not need to be restarted through PM2. It is compiled into static files and served by nginx.

26. Verify nginx configuration

Check nginx configuration syntax:

```
sudo nginx -t
```

Expected result:

```
syntax is ok  
test is successful
```

Reload nginx:

```
sudo systemctl reload nginx
```

A reload is usually sufficient. A full restart is not normally necessary.

Check nginx status:

```
sudo systemctl status nginx --no-pager
```

27. Verify the client deployment

Test the website headers:

```
curl -I https://draftproanalytics.com
```

Expected result should normally include:

HTTP/2 200

Open the application in a browser and verify:

1. The login page loads.
2. Static JavaScript and CSS files load without 404 errors.
3. Browser console has no fatal errors.
4. API requests reach /api.
5. Authentication works.
6. The newly deployed feature works.
7. Refreshing a Vue Router route does not produce an nginx 404.

Part VI — Recommended deployment order

Deploy releases in this order when both server and client changed:

1. Merge server feature branch into server main.
2. Push server main to GitHub.
3. Merge client feature branch into client main.
4. Push client main to GitHub.
5. SSH into the VPS.
6. Deploy and build the server.
7. Restart and verify the PM2 server process.
8. Deploy and build the client.
9. Reload nginx.
10. Perform browser and API verification.

Deploying the server first reduces the chance that the new client will call API endpoints that have not yet been installed.

Part VII — Recommended deployment commands

Server command sequence

```
ssh <vps-user>@104.225.219.35

cd /var/www/draftproanalytics-server

git status
git fetch origin
git switch main
```



```
git pull --ff-only origin main

npm ci
npx prisma generate
npm run build

pm2 restart draftproanalytics-server --update-env
pm2 save
pm2 status
pm2 logs draftproanalytics-server --lines 100
```

Client command sequence

```
cd /var/www/draftproanalytics-client

git status
git fetch origin
git switch main
git pull --ff-only origin main

npm ci
npm run build

sudo nginx -t
sudo systemctl reload nginx
sudo systemctl status nginx --no-pager

curl -I https://draftproanalytics.com
```

Part VIII — Emergency rollback

28. Find the previous Git commit

```
git log --oneline -10
```

Identify the last known working commit.

29. Roll back the server

```
cd /var/www/draftproanalytics-server

git reset --hard <previous-commit-hash>
npm ci
npx prisma generate
npm run build
pm2 restart draftproanalytics-server --update-env
```

Do not roll back database changes blindly. Database rollback requires a separately reviewed MySQL rollback script.

30. Roll back the client

```
cd /var/www/draftproanalytics-client

git reset --hard <previous-commit-hash>
npm ci
npm run build
sudo nginx -t
sudo systemctl reload nginx
```

Part IX — `.gitignore` requirements

Both repositories should include:

```
.env
.env.*
!.env.example
!.env.*.example
```

This ignores real environment files while allowing example environment files to be committed.

Before every push, verify:

```
git status  
git ls-files | grep -E '(^|/)\.env($|\.)'
```

Only example environment files should appear.